

머신러닝 프로그래밍

Machine Learning Programming

Lecture 8 : Support Vector Machine



충북대학교
CHUNGBUK NATIONAL UNIVERSITY



충북대학교
산업인공지능연구센터

Support Vector Machines

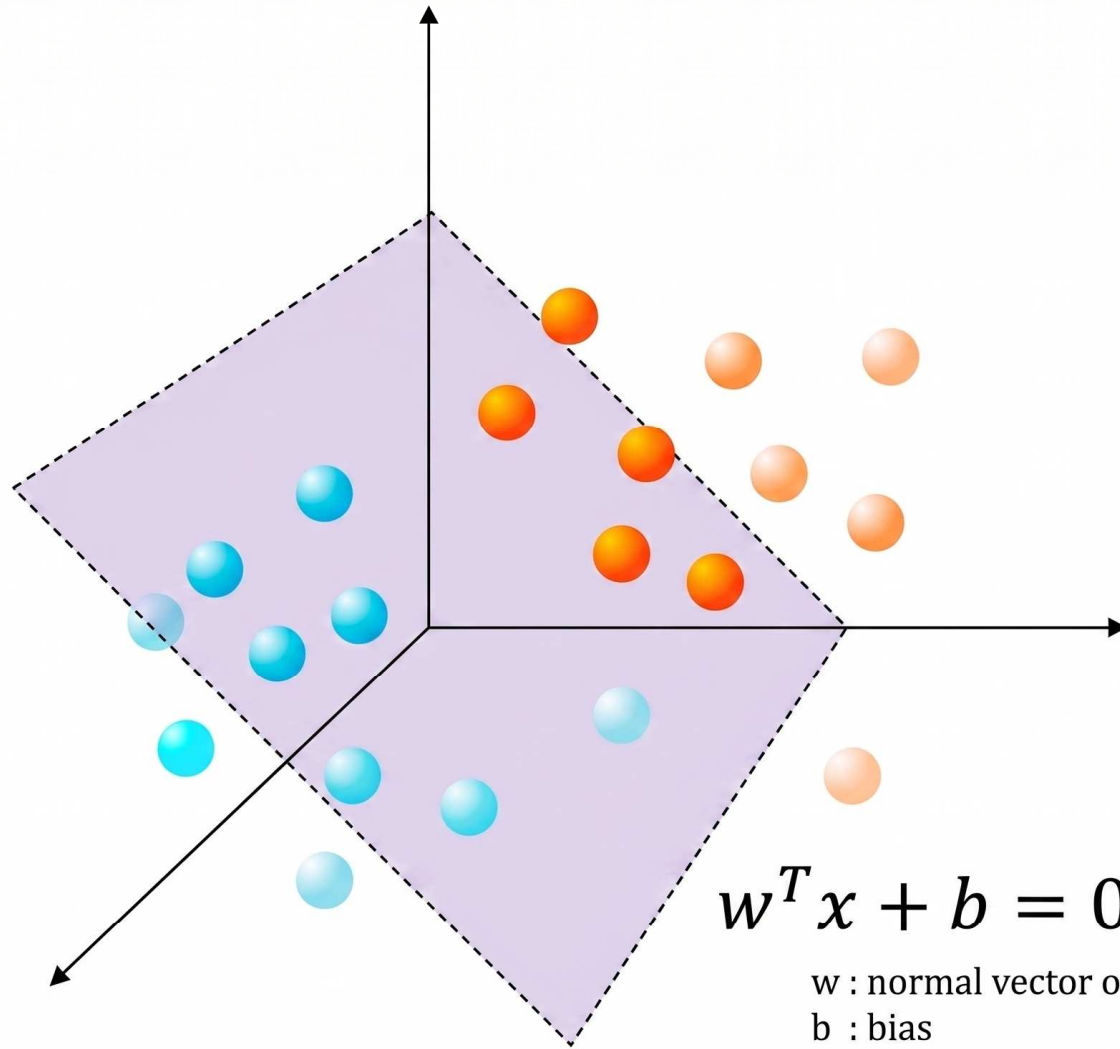
- 서포트 벡터 머신 모델 개요
- 선형 분리 가능한 경우를 위한 선형 SVM
- 선형 분리 불가능한 경우를 위한 선형 SVM
- 비선형 SVM

Support Vector Machines (SVM)

특징

- 일반적으로 훈련 데이터에 대한 분류 성능 극대화 노력.
- 분류기가 훈련 데이터에 너무 과도하게 맞춰지면(과적합), 일반화 능력 저하.
- **직접 결정 함수가 일반화 능력을 극대화하도록 훈련.**
- **통계적 학습 이론(statistical learning theory) 기반.**

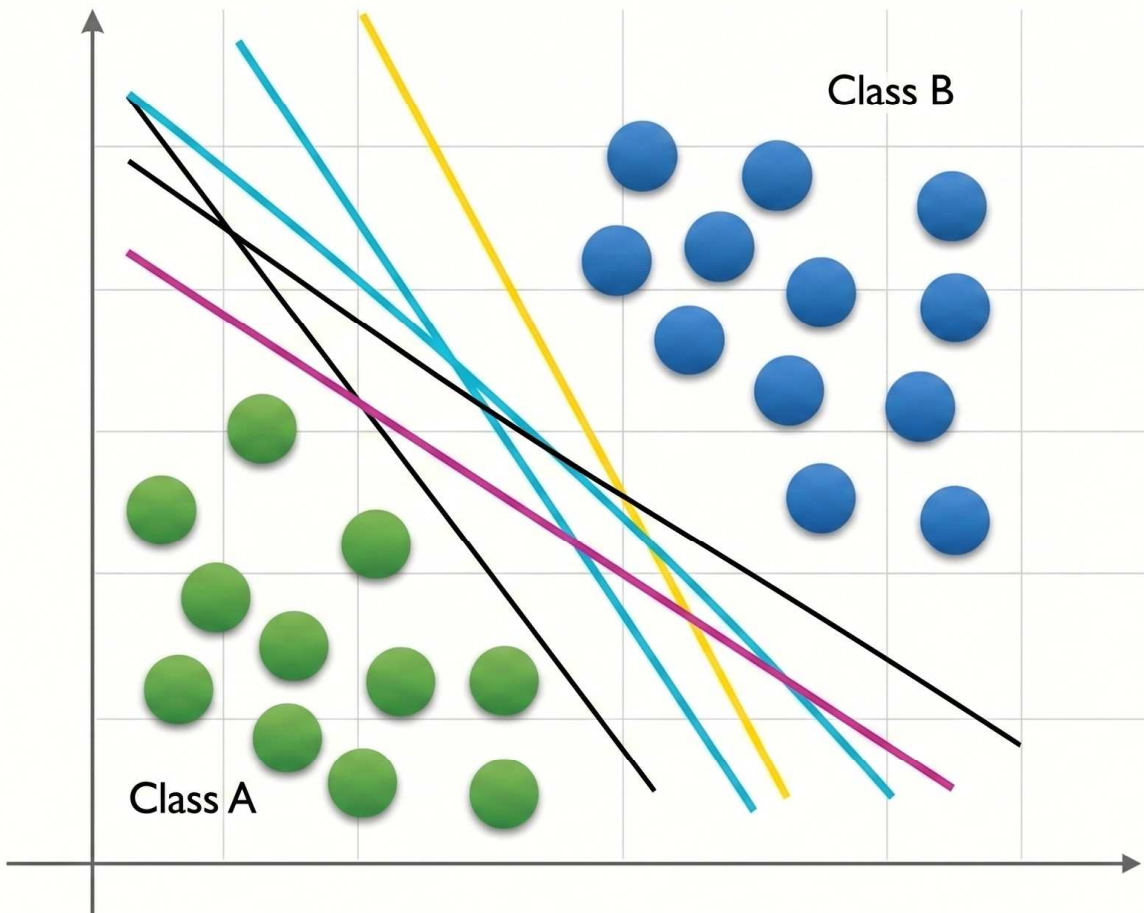
Separating Hyperplane



$$w^T x + b = 0$$

w : normal vector of the hyperplane
 b : bias

Separating Hyperplane



Two class classification 문제

두 class를 나누는 hyperplane은
무한히 많음

어떤 hyperplane이 가장 “좋은”
hyperplane 인가?

“좋다”는 것의 기준은?

Separating Hyperplane

Maximizing margin over the training set

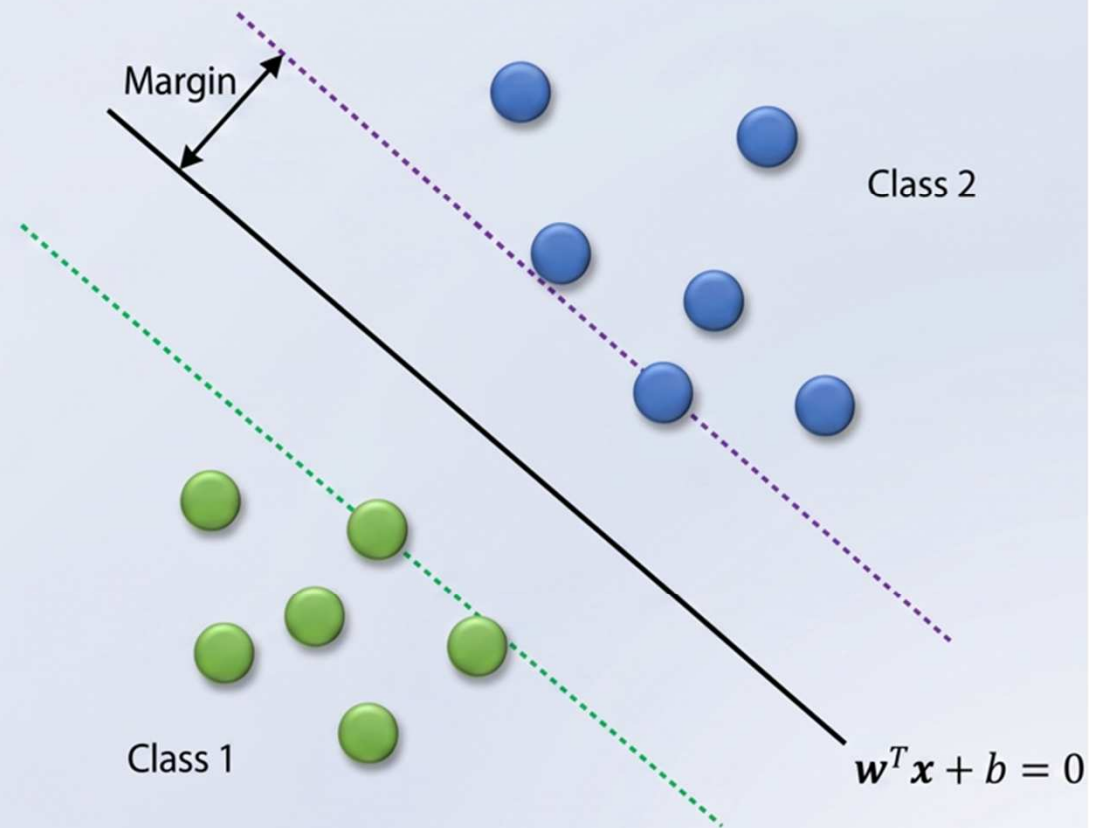
= minimizing generalization error

= good prediction performance

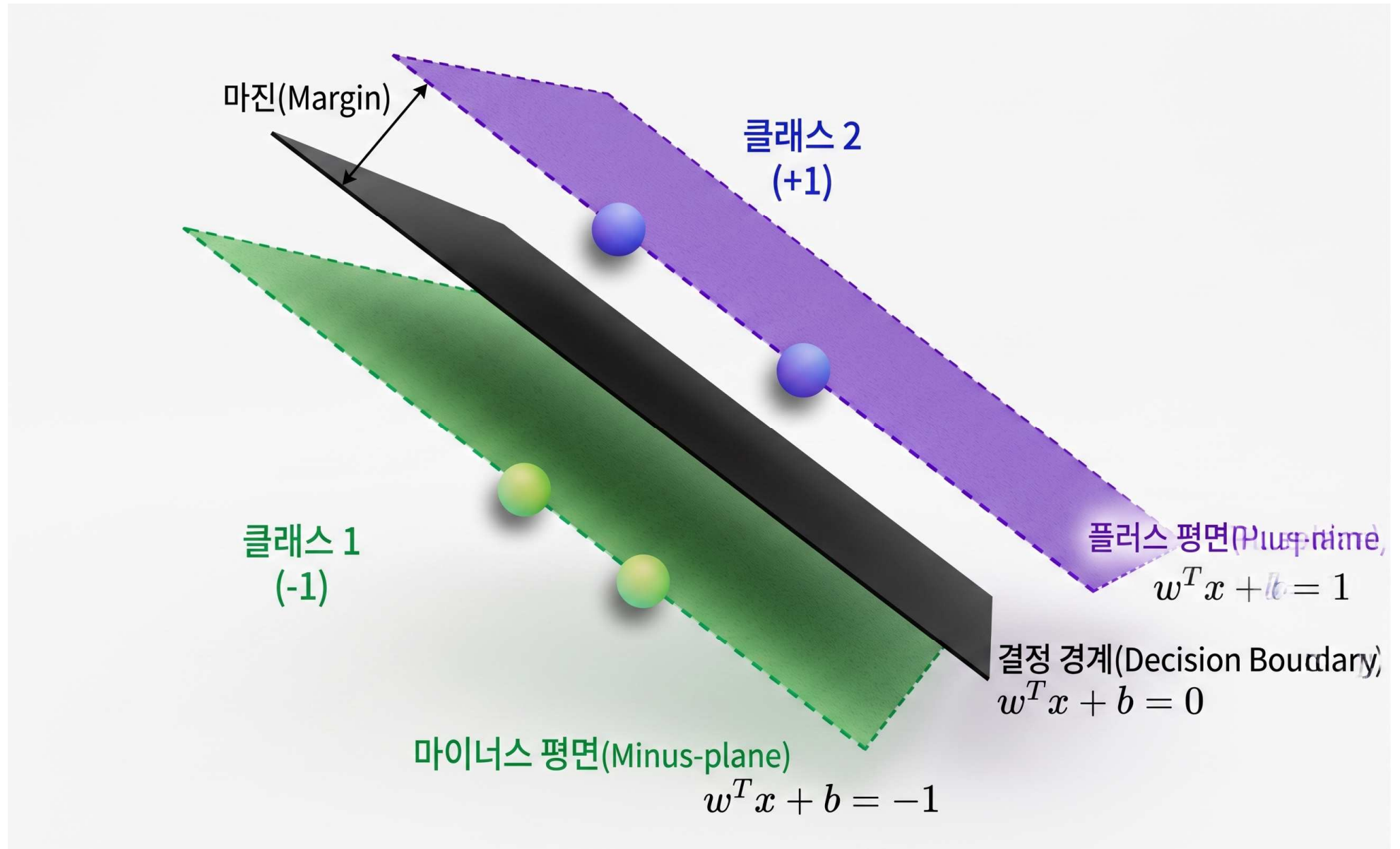
So, what is the **margin**?

서포트 벡터 머신 (SVM): 마진(Margin)의 최적화

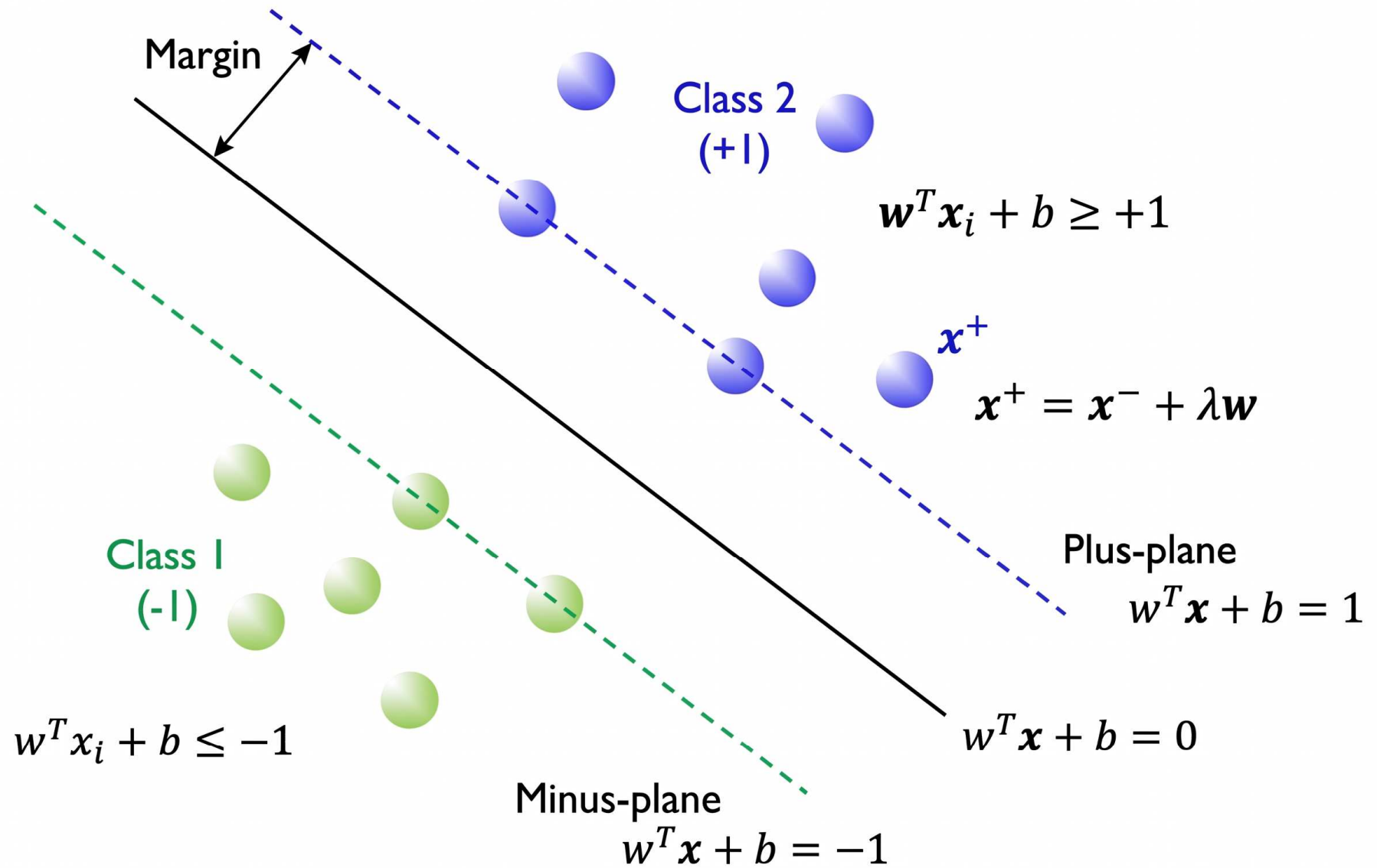
- 마진(Margin)의 정의:
 - 각 클래스에서 가장 가까운 관측치(서포트 벡터) 사이의 거리.
 - 결정 경계로부터의 최대 거리 확보가 목표.
- 마진의 표현:
 - 마진은 법선 벡터 $\mathbf{w}$ (기울기 정보 포함) 로 표현 가능.
 - (예: 마진 = $2 / \|\mathbf{w}\|$, 여기서 $\|\mathbf{w}\|$ 는 법선 벡터의 크기).



Geometric Margin



Geometric Margin



Geometric Margin

$$w^T x^+ + b = 1 \quad x^+ \text{가 plus-plane 위의 점}$$

$$w^T (x^- + \lambda w) + b = 1 \quad (x^+ = x^- + \lambda w)$$

$$w^T x^- + b + \lambda w^T w = 1$$

$$-1 + \lambda w^T w = 1 \quad x^- \text{는 minus-plane 위의 점}$$

$$\lambda = \frac{2}{w^T w}$$

Geometric Margin

$$w^T x^+ + b = 1 \quad x^+ \text{가 plus-plane 위의 점}$$

$$w^T (x^- + \lambda w) + b = 1 \quad (x^+ = x^- + \lambda w)$$

$$w^T x^- + b + \lambda w^T w = 1$$

$$-1 + \lambda w^T w = 1 \quad x^- \text{는 minus-plane 위의 점}$$

$$\lambda = \frac{2}{w^T w}$$

The vector norm $\|W\|_p$ for $p = 1, 2, 3, \dots$

$$\|W\|_p = \left(\sum_i |w_i|^p \right)^{1/p} \quad \text{L}_2 \text{ norm} \quad \|W\|_2 = \left(\sum_i |w_i|^2 \right)^{1/2} = \sqrt{w_1^2 + w_2^2 + \dots + w_n^2} = \sqrt{W^T W}$$

$$\begin{aligned} \text{Margin} &= \text{distance}(x^+, x^-) \\ &= \|x^+ - x^-\|_2 \\ &= \|(x^- + \lambda w) - x^-\|_2 \\ &= \|\lambda w\|_2 \\ &= \lambda \sqrt{w^T w} \\ &= \frac{2}{w^T w} \cdot \sqrt{w^T w} \\ &= \frac{2}{\sqrt{w^T w}} = \frac{2}{\|w\|_2} \end{aligned}$$

Geometric Margin

$$\max \text{Margin} = \max \frac{2}{\|w\|_2} \Leftrightarrow \min \frac{1}{2} \|w\|_2$$

w의 L_2 norm이 제곱근을 포함하고 있기 때문에 계산이 어려움
→ 계산상의 편의를 위해 다음과 같은 형태로 목적함수를 변경

$$\min \frac{1}{2} \|w\|_2 \Leftrightarrow \min \frac{1}{2} \|w\|_2^2$$

Convex Optimization Problem

$$\text{minimize } \frac{1}{2} \|w\|_2^2$$

$$\text{subject to } y_i(w^T x_i + b) \geq 1, i = 1, 2, \dots, n$$

- Decision variable은 w 와 b
- Objective function은 separating hyperplane으로 부터 정의된margin의 역수
- Constraint는 training data를 완벽하게 separating하는 조건
- Objective function is quadratic and constraint is linear $\rightarrow$ quadratic programming
 $\rightarrow$ convex optimization $\rightarrow$ globally optimal solution exists
(전역최적해 존재)
- Training data가 linearly separable한 경우에만 해가 존재함

Lagrangian Formulation

Original Problem

$$\text{minimize } \frac{1}{2} \|w\|_2^2$$

$$\text{subject to } y_i(w^T x_i + b) \geq 1, i = 1, 2, \dots, n$$

Lagrangian multiplier를 이용하여 Lagrangian primal문제로 변환

Lagrangian Primal

$$\max_{\alpha} \min_{w, b} \mathcal{L}(w, b, \alpha) = \frac{1}{2} \|w\|_2^2 - \sum_{i=1}^n \alpha_i (y_i(w^T x_i + b) - 1)$$

$$\text{subject to } \alpha_i \geq 0, i = 1, 2, \dots, n$$

Lagrangian Formulation

Lagrangian Primal

$$\max_{\alpha} \min_{w, b} \mathcal{L}(w, b, \alpha) = \frac{1}{2} \|w\|_2^2 - \sum_{i=1}^n \alpha_i (y_i (w^T x_i + b) - 1)$$

subject to $\alpha_i \geq 0, i = 1, 2, \dots, n$

Convex, continuous이기 때문에 미분 = 0에서 최소값을 가짐

$$\textcircled{1} \quad \frac{\partial \mathcal{L}(w, b, \alpha)}{\partial w} = 0 \quad \longrightarrow \quad w = \sum_{i=1}^n \alpha_i y_i x_i$$

$$\textcircled{2} \quad \frac{\partial \mathcal{L}(w, b, \alpha)}{\partial b} = 0 \quad \longrightarrow \quad \sum_{i=1}^n \alpha_i y_i = 0$$

Lagrangian Formulation

$$\underbrace{\frac{1}{2} \|w\|_2^2}_{\textcircled{1}} - \underbrace{\sum_{i=1}^n \alpha_i (y_i (w^T x_i + b) - 1)}_{\textcircled{2}}$$

$$\textcircled{1} \quad \frac{1}{2} \|w\|_2^2 = \frac{1}{2} w^T w$$

$$= \frac{1}{2} w^T \sum_{j=1}^n \alpha_j y_j x_j$$

$$= \frac{1}{2} \sum_{j=1}^n \alpha_j y_j (w^T x_j)$$

$$= \frac{1}{2} \sum_{j=1}^n \alpha_j y_j \left(\sum_{i=1}^n \alpha_i y_i x_i^T x_j \right)$$

$$= \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j x_i^T x_j$$

$$\boxed{\frac{\partial \mathcal{L}(w, b, \alpha)}{\partial w} = 0 \implies w = \sum_{i=1}^n \alpha_i y_i x_i}$$

Lagrangian Formulation

$$\mathcal{L}(w, b, \alpha) = \underbrace{\frac{1}{2} \|w\|_2^2}_{\textcircled{1}} - \underbrace{\sum_{i=1}^n \alpha_i (y_i (w^T x_i + b) - 1)}_{\textcircled{2}}$$

$$\textcircled{2} \quad - \sum_{i=1}^n \alpha_i (y_i (w^T x_i + b) - 1)$$

$$= - \sum_{i=1}^n \alpha_i y_i (w^T x_i + b) + \sum_{i=1}^n \alpha_i$$

$$= - \sum_{i=1}^n \alpha_i y_i w^T x_i - b \sum_{i=1}^n \alpha_i y_i + \sum_{i=1}^n \alpha_i$$

$$= - \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j x_i^T x_j + \sum_{i=1}^n \alpha_i$$

$$\begin{aligned} \frac{\partial \mathcal{L}(w, b, \alpha)}{\partial b} = 0 &\rightarrow \sum_{i=1}^n \alpha_i y_i = 0 \\ \frac{\partial \mathcal{L}(w, b, \alpha)}{\partial w} = 0 &\rightarrow w = \sum_{i=1}^n \alpha_i y_i x_i \end{aligned}$$

Lagrangian Dual

$$\min_{w,b} \frac{1}{2} \|w\|_2^2 - \sum_{i=1}^n \alpha_i (y_i (w^T x_i + b) - 1)$$

$$= \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j x_i^T x_j$$

$$\text{where } \sum_{i=1}^n \alpha_i y_i = 0$$

Lagrangian Dual

따라서 Lagrangian dual은 다음과 같은 quadratic programming formulation

$$\underset{\alpha}{\text{maximize}} \sum_{i=1}^n \alpha_i - \sum_{i=1}^n \sum_{j=1}^n \overbrace{\alpha_i \alpha_j y_i y_j}^{\text{quadratic}} \underbrace{x_i^T x_j}_{\text{Inner product}}$$

$$\text{subject to } \overbrace{\sum_{i=1}^n \alpha_i y_i}^{\text{linear}} = 0,$$

$$\alpha_i \geq 0, \quad i = 1, 2, \dots, n$$

- Original problem formulation (primal formulation) 보다 풀기 쉬운 형태
- Objective function is quadratic and constraint is linear → quadratic programming → convex optimization → globally optimal solution exists (전역최적해 존재)
- Optimization 문제가 x들의 inner product만으로 표현됨 (nonlinear case로 확장했을 때 좋은 성질)
- Lagrangian dual의 decision variable은 α 이며, quadratic optimization을 풀어 α 에 대한 solution을 얻을 수 있음 $\alpha^* = (\alpha_1^*, \alpha_2^*, \dots, \alpha_n^*)$

Lagrangian Dual

(w, b, α) 가 Lagrangian dual problem의 최적해가 되기 위한 조건

KKT (Karush-Kuhn-Tucker) conditions:

① Stationarity

$$\frac{\partial \mathcal{L}(w, b, \alpha)}{\partial w} = 0 \Rightarrow w = \sum_{i=1}^n \alpha_i y_i x_i \quad \frac{\partial \mathcal{L}(w, b, \alpha)}{\partial b} = 0 \Rightarrow \sum_{i=1}^n \alpha_i y_i = 0$$

② Primal feasibility $y_i(w^T x_i + b) \geq 1, i = 1, 2, \dots, n$

③ Dual feasibility $\alpha_i \geq 0, i = 1, 2, \dots, n$

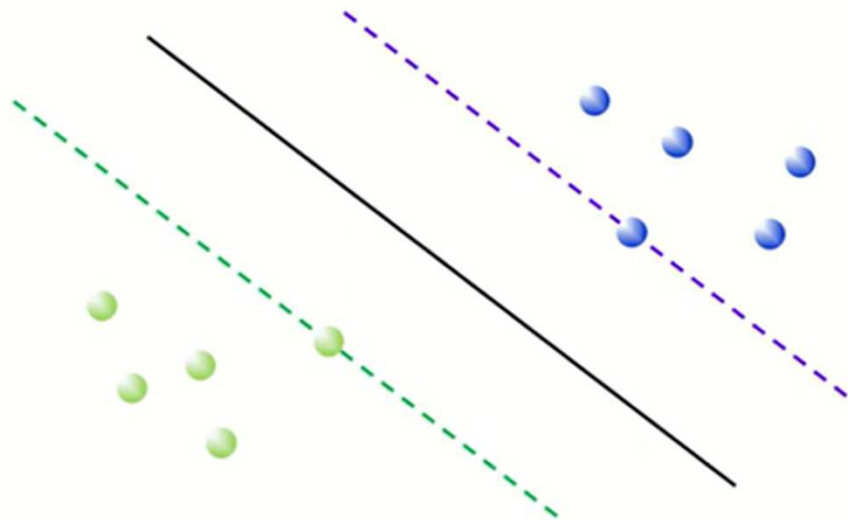
④ Complementary slackness $\alpha_i(y_i(w^T x_i + b) - 1) = 0$

Characteristics of the Solutions

$$\alpha_i(y_i(w^T x_i + b) - 1) = 0, i = 1, 2, \dots, n$$

① $\alpha_i > 0$ and $y_i(w^T x_i + b) - 1 = 0$ x_i 가 plus-plane 또는 minus-plane (마진) 위에 있음
(**support vector**) 해당 $\alpha_i > 0$

② $\alpha_i = 0$ and $y_i(w^T x_i + b) - 1 \neq 0$ x_i 가 plus-plane 또는 minus-plane 위에 있지 않음
해당 $\alpha_i = 0$
Hyperplane을 구축하는데 영향을 미치지 않음
SVM이 outlier에 robust (강건)한 이유



Characteristics of the Solutions

x_i 가 support vector인 경우에만 $\alpha_i^* \geq 0$ 이므로

$$w^* = \sum_{i=1}^n \alpha_i^* y_i x_i = \sum_{i \in SV} \alpha_i^* y_i x_i$$

즉, support vector만 이용하여 optimal hyperplane (decision boundary) 을 구할 수 있다
(sparse representation!)

또한, 다음과 같이 임의의 support vector 하나를 이용하여 b^* 를 구할 수 있다.

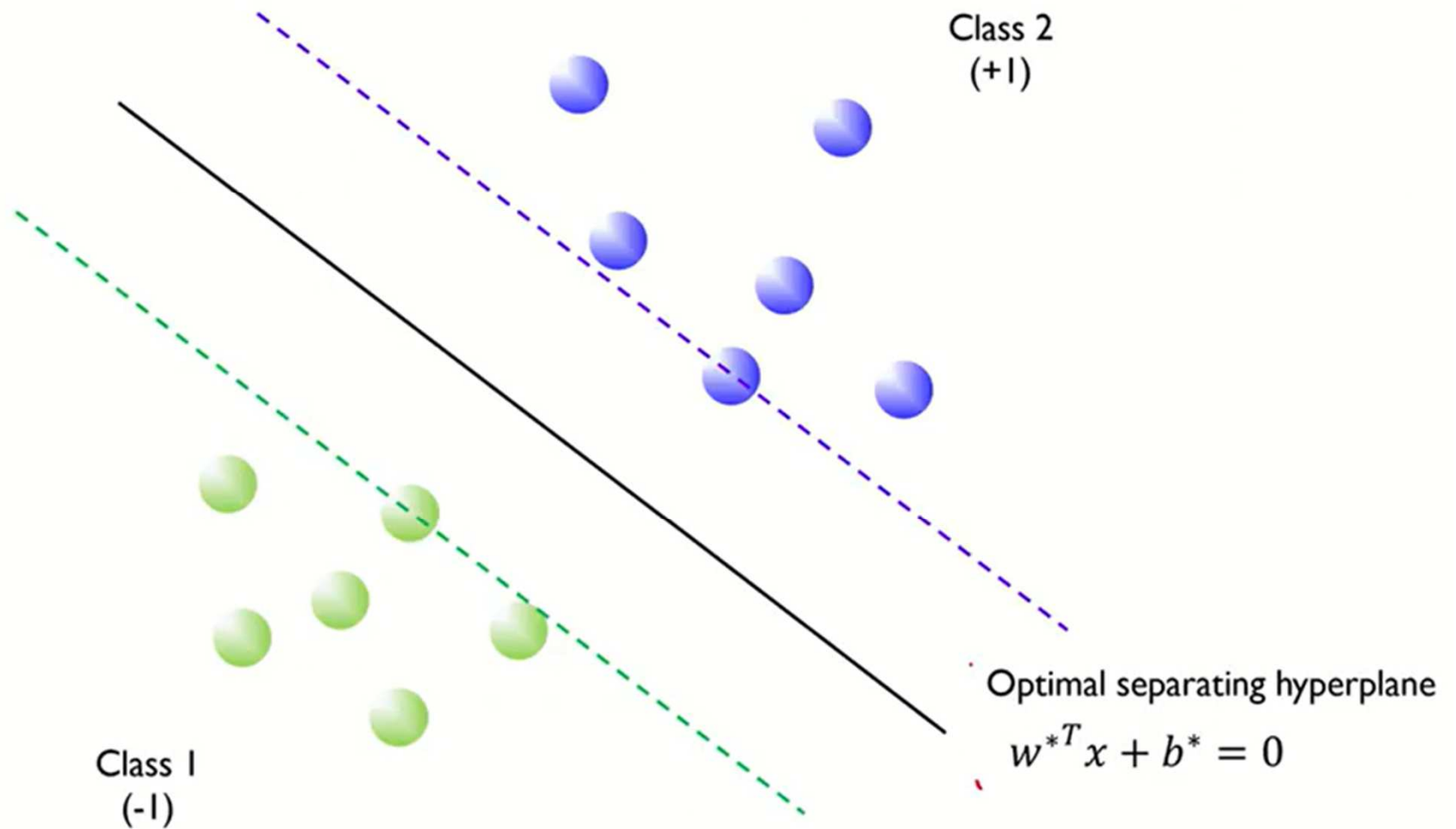
$$w^{*T} + b^* = y_{sv}$$

$$w^{*T} + b^* = \sum_{i=1}^n \alpha_i^* y_i x_i^T x_{sv} + b^* = y_{sv}$$

$$b^* = y_{sv} - \sum_{i=1}^n \alpha_i^* y_i x_i^T x_{sv}$$

Classifying New Data Points

Training data로부터 w^* 와 b^* 를 계산하였다면, 새로운 데이터에 대한 분류를 시행할 수 있음



Classifying New Data Points

새로운 데이터가 optimal separating hyperplane보다 밑에 있음

$$w^{*T}x_{new} + b^* < 0$$

→ class 1 (-1) 로 예측 $\hat{y}_{new} = -1$

새로운 데이터가 optimal separating hyperplane보다 위에 있음

$$w^{*T}x_{new} + b^* > 0$$

→ class 2 (+1) 로 예측 $\hat{y}_{new} = +1$

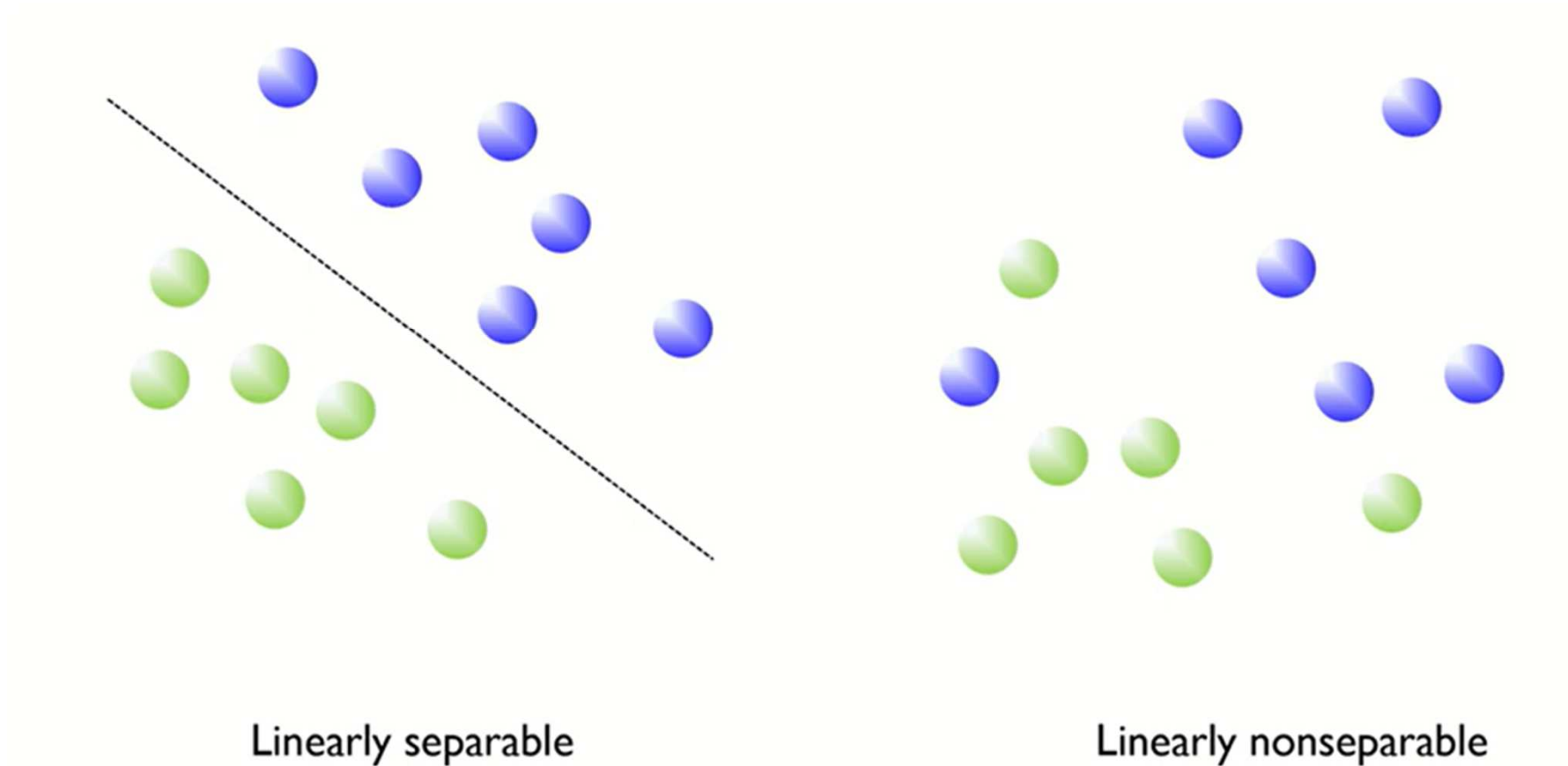
따라서 다음과 같이 새로운 데이터에 class를 부여할 수 있음

$$\hat{y}_{new} = \text{sign}(w^{*T}x_{new} + b^*)$$

$$= \text{sign}\left(\sum_{i \in SV} \alpha_i^* y_i x_i^T x_{new} + b^*\right)$$

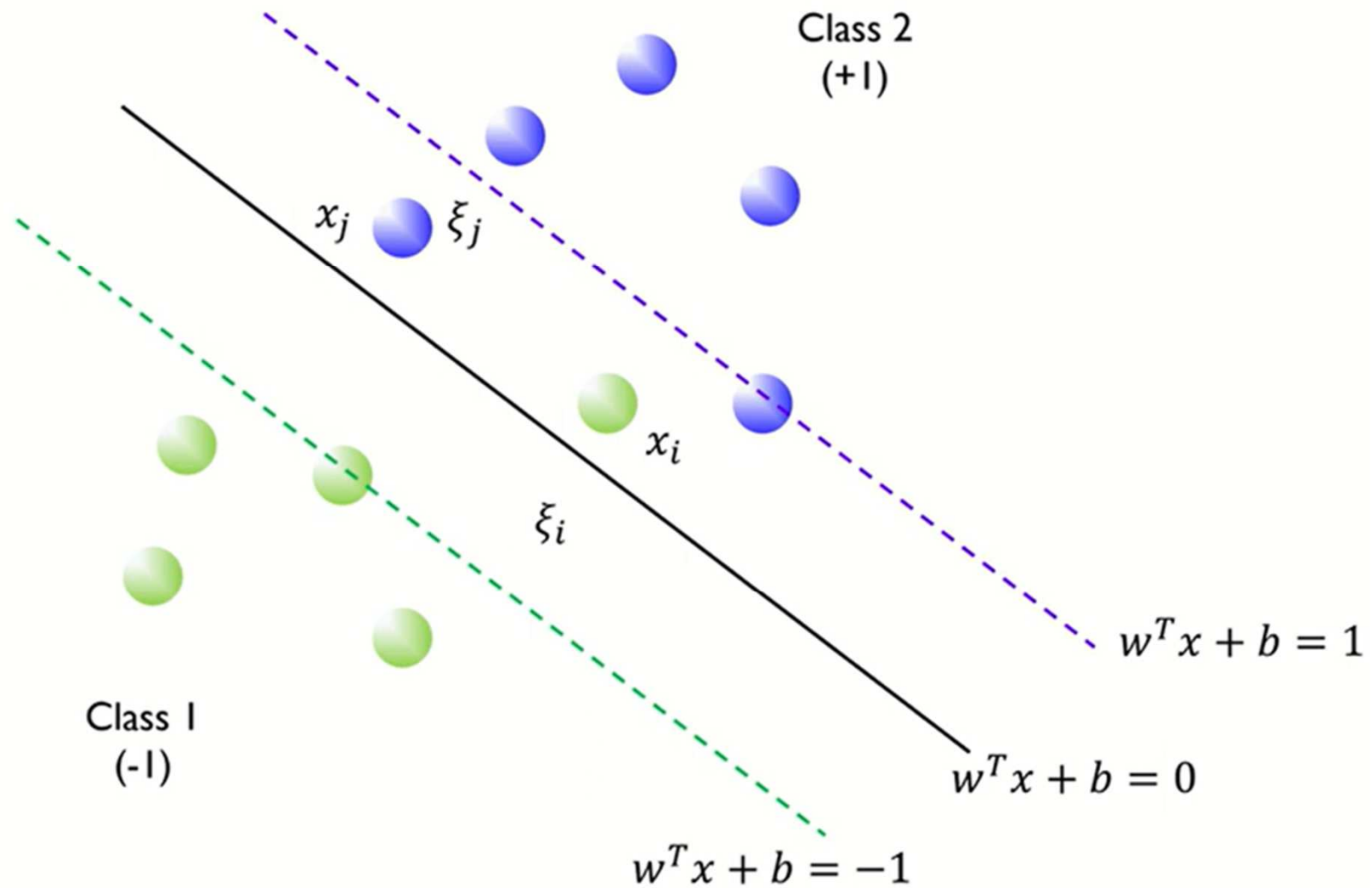
Linearly Nonseparable Case (Soft Margin SVM)

Linearly Nonseparable Problems



Linear decision boundary를 이용하여 완벽하게 나누는 것은 불가능
→ Error 허용

Linearly Nonseparable Problems



Convex Optimization Formulation

$$\underset{w,b,\xi}{\text{minimize}} \quad \frac{1}{2} \|w\|_2^2 + C \sum_{i=1}^n \xi_i$$

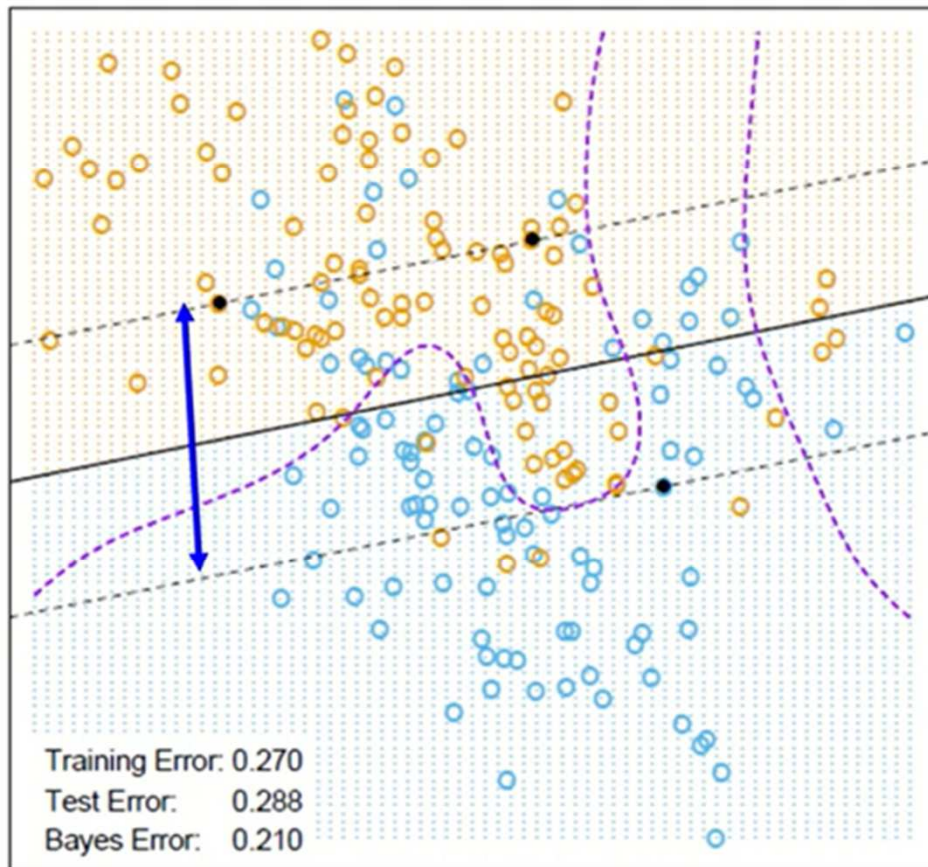
$$\text{subject to } y_i(w^T x_i + b) \geq 1 - \xi_i, \xi_i \geq 0, i = 1, 2, \dots, n$$

- Decision variable은 w, b, ξ
- Slack variable $\xi_i \geq 0$ 도입하여 training error 허용 $\rightarrow$ 그렇다고 마냥 크게 할 수 없음
- Objective function에 penalty 를 추가하여 억제
- C 는 margin과 training error에 대한 trade-off를 결정하는 tuning parameter
 - $C \uparrow$: training error를 많이 허용하지 않음 $\rightarrow$ overfit
 - $C \downarrow$: training error 많이 허용 $\rightarrow$ underfit
- Training data가 linearly separable하지 않아도 해가 존재함

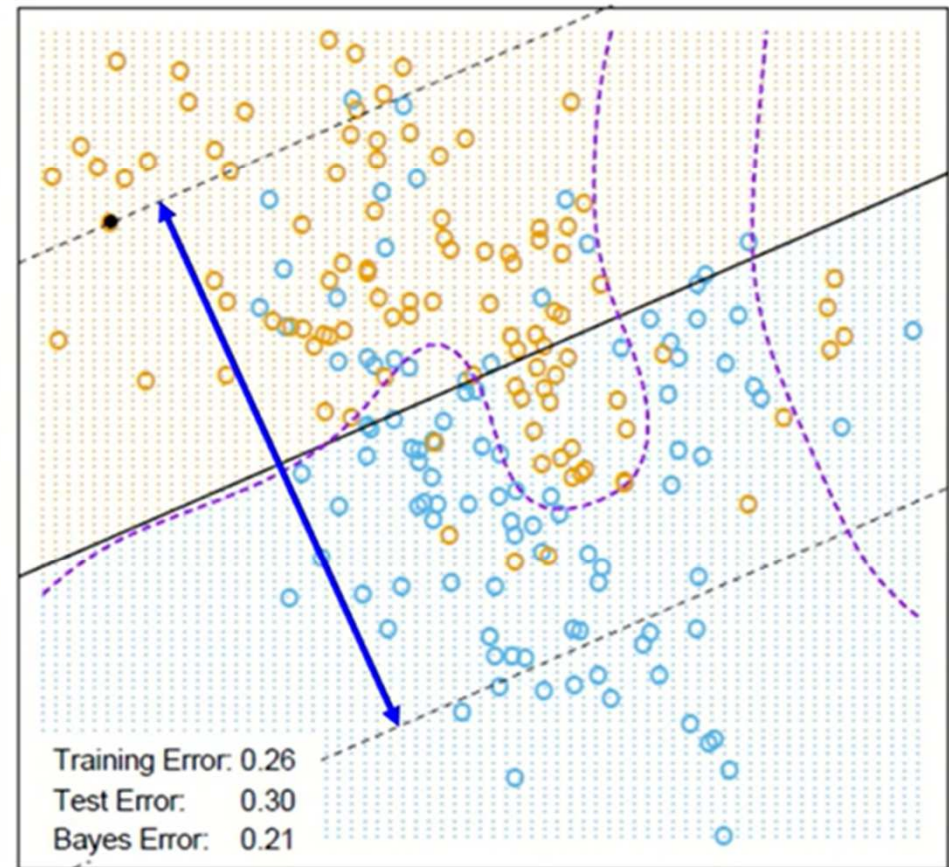
Soft Margin SVM Classifiers

$C \uparrow$: training error를 많이 허용하지 않음 $\rightarrow$ overfit

$C \downarrow$: training error 많이 허용 $\rightarrow$ underfit



$C = 10000$



$C = 0.01$

Lagrangian Formulation

Original Problem

$$\text{minimize } \frac{1}{2} \|w\|_2^2 + C \sum_{i=1}^n \xi_i$$

$$\text{subject to } y_i(w^T x_i + b) \geq 1 - \xi_i, i = 1, 2, \dots, n$$

Lagrangian multiplier를 이용하여 Lagrangian primal 문제로 변환

Lagrangian Primal

$$\max_{\alpha, \gamma} \min_{w, b, \xi} \mathcal{L}(w, b, \alpha, \xi, \gamma) = \frac{1}{2} \|w\|_2^2 + C \sum_{i=1}^n \xi_i - \sum_{i=1}^n \alpha_i (y_i(w^T x_i + b) - 1 + \xi_i) - \sum_{i=1}^n \gamma_i \xi_i$$

$$\text{subject to } \alpha_i \geq 0, \gamma_i \geq 0, i = 1, 2, \dots, n$$

Lagrangian Formulation

Lagrangian Primal

$$\max_{\alpha, \gamma} \min_{w, b, \xi} \mathcal{L}(w, b, \alpha, \xi, \gamma) = \frac{1}{2} \|w\|_2^2 + C \sum_{i=1}^n \xi_i - \sum_{i=1}^n \alpha_i (y_i (w^T x_i + b) - 1 + \xi_i) - \sum_{i=1}^n \gamma_i \xi_i$$

subject to $\alpha_i \cdot \gamma_i \geq 0, i = 1, 2, \dots, n$

Convex, continuous이기 때문에 미분 = 0 에서 최소값을 가짐

1	$\frac{\partial \mathcal{L}(w, b, \xi, \alpha, \gamma)}{\partial w} = 0$	$\implies$	$w = \sum_{i=1}^n \alpha_i y_i x_i$
2	$\frac{\partial \mathcal{L}(w, b, \xi, \alpha, \gamma)}{\partial b} = 0$	$\implies$	$\sum_{i=1}^n \alpha_i y_i = 0$
3	$\frac{\partial \mathcal{L}(w, b, \xi, \alpha, \gamma)}{\partial \xi_i} = 0$	$\implies$	$C - \alpha_i - \gamma_i = 0, i = 1, 2, \dots, n$

Lagrangian Dual

$$\begin{aligned}\mathcal{L}(w, b, \alpha, \xi, \gamma) &= \frac{1}{2} \|w\|_2^2 + C \sum_{i=1}^n \xi_i - \sum_{i=1}^n \alpha_i (y_i (w^T x_i + b) - 1 + \xi_i) - \sum_{i=1}^n \gamma_i \xi_i \\&= \frac{1}{2} \|w\|_2^2 - \sum_{i=1}^n \alpha_i (y_i (w^T x_i + b) - 1) + C \sum_{i=1}^n \xi_i - \sum_{i=1}^n \alpha_i \xi_i - \sum_{i=1}^n \gamma_i \xi_i \\&= \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j x_i^T x_j + C \sum_{i=1}^n \xi_i - \sum_{i=1}^n (\alpha_i - \gamma_i) \xi_i \\&= \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j x_i^T x_j + C \sum_{i=1}^n \xi_i - C \sum_{i=1}^n \xi_i \\ \mathcal{L}(w, b, \alpha) &= \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j x_i^T x_j\end{aligned}$$

$$\frac{\partial \mathcal{L}(w, b, \xi, \alpha, \gamma)}{\partial \xi_i} = 0 \implies C - \alpha_i - \gamma_i = 0, i = 1, 2, \dots, n$$

Lagrangian Dual

$$\min_{w,b,\xi} \frac{1}{2} \|w\|_2^2 + C \sum_{i=1}^n \xi_i - \sum_{i=1}^n \alpha_i (y_i (w^T x_i + b) - 1 + \xi_i) - \sum_{i=1}^n \gamma_i \xi_i$$

$$= \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j x_i^T x_j$$

$$\text{where } \sum_{i=1}^n \alpha_i y_i = 0 \text{ and } C - \alpha_i - \gamma_i = 0, i = 1, 2, \dots, n$$

Note : $\alpha_i \geq 0, \gamma_i \geq 0$, and $C - \alpha_i - \gamma_i = 0$

$$\implies 0 \leq \alpha_i \leq C$$

Lagrangian Dual

따라서 Lagrangian dual은 다음과 같음

Soft Margin SVM

$$\begin{aligned} & \underset{\alpha}{\text{maximize}} \quad \frac{1}{2} \|w\|_2^2 - \sum_{i=1}^n \alpha_i \alpha_j y_i y_j x_i^T x_j \\ & \text{subject to} \quad \sum_{i=1}^n \alpha_i y_i = 0 \\ & \quad \quad \quad 0 \leq \alpha_i \leq C, i = 1, 2, \dots, n \end{aligned}$$

Hard Margin SVM

$$\begin{aligned} & \underset{\alpha}{\text{maximize}} \quad \frac{1}{2} \|w\|_2^2 - \sum_{i=1}^n \alpha_i \alpha_j y_i y_j x_i^T x_j \\ & \text{subject to} \quad \sum_{i=1}^n \alpha_i y_i = 0 \\ & \quad \quad \quad 0 \leq \alpha_i, i = 1, 2, \dots, n \end{aligned}$$

Linearly separable case와 마찬가지로 decision variable은 α 이며,
quadratic programming을 풀어 α 에 대한 solution을 얻을 수 있음

Lagrangian Dual

$(w, b, \xi, \alpha, \gamma)$ 가 Lagrangian dual problem의 해가 되기 위한 조건 (KKT condition)

$$\textcircled{1} \quad \frac{\partial \mathcal{L}(w, b, \xi, \alpha, \gamma)}{\partial w} = 0 \quad \Longrightarrow \quad w = \sum_{i=1}^n \alpha_i y_i x_i$$

$$\textcircled{2} \quad \frac{\partial \mathcal{L}(w, b, \xi, \alpha, \gamma)}{\partial b} = 0 \quad \Longrightarrow \quad \sum_{i=1}^n \alpha_i y_i = 0$$

$$\textcircled{3} \quad \frac{\partial \mathcal{L}(w, b, \xi, \alpha, \gamma)}{\partial \xi_i} = 0 \quad \Longrightarrow \quad C - \alpha_i - \gamma_i = 0, \quad i = 1, 2, \dots, n$$

Complementary slackness

$$\textcircled{4} \quad \alpha_i (y_i (w^T x_i + b) - 1 + \xi_i) = 0, \quad \gamma_i \xi_i = 0, \quad i = 1, 2, \dots, n$$

Characteristics of the Solution

KKT condition으로부터 다음과 같은 정보를 얻을 수 있음:

$$\alpha_i(y_i(w^T x_i + b) - 1 + \xi_i) = 0,$$

$$\alpha_i = C - \gamma_i, \quad \gamma_i \xi_i = 0, i = 1, 2, \dots, n$$

① $\alpha_i = 0 \Rightarrow \gamma_i = C$

$$\Rightarrow \xi_i = 0$$

$$\Rightarrow (y_i(w^T x_i + b) - 1) \neq 0$$

$\Rightarrow x_i$ 가 plus-plane 또는 minus-plane 위에 있지 않음

② $0 < \alpha_i < C \Rightarrow \gamma_i > 0$

$$\Rightarrow \xi_i = 0, \gamma_i \xi_i = 0$$

$$\Rightarrow (y_i(w^T x_i + b) - 1) = 0$$

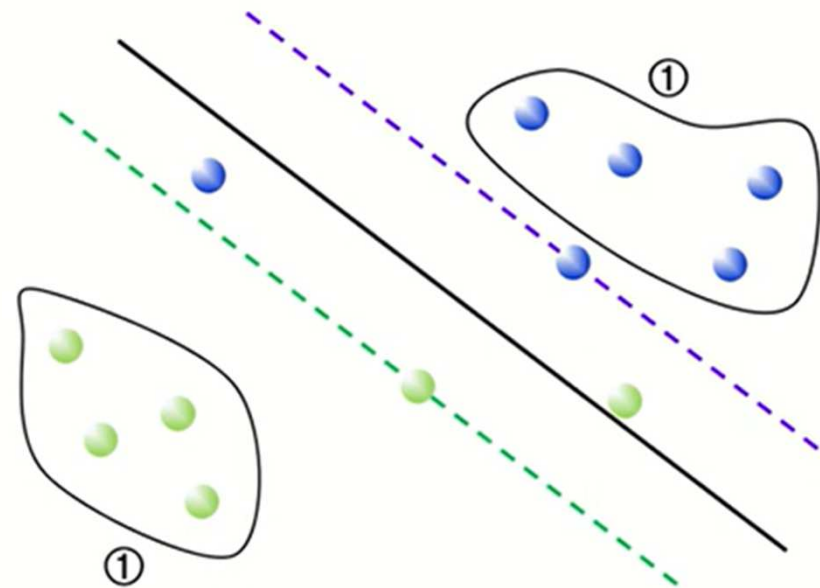
$\Rightarrow x_i$ 가 plus-plane 또는 minus-plane 위에 있음
(**support vector**)

③ $\alpha_i = C \Rightarrow \gamma_i = 0$

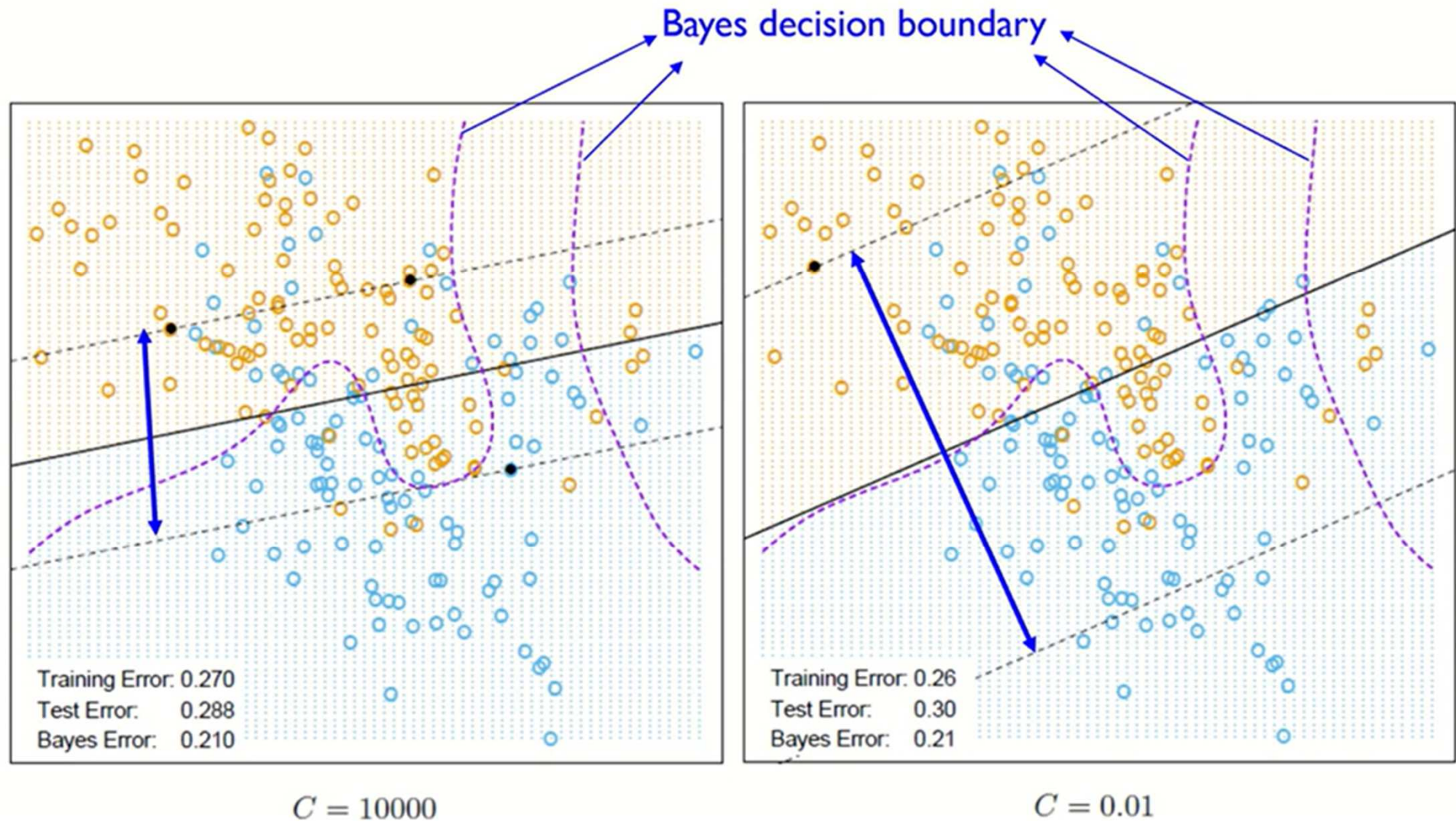
$$\Rightarrow \xi_i > 0$$

$$\Rightarrow \alpha_i(y_i(w^T x_i + b) - 1) = -\alpha_i \xi_i \neq 0$$

$\Rightarrow x_i$ 가 plus-plane과 minus-plane 사이에 있음
(**support vector**)



Soft Margin SVM Classifiers



Soft Margin SVM Classifiers

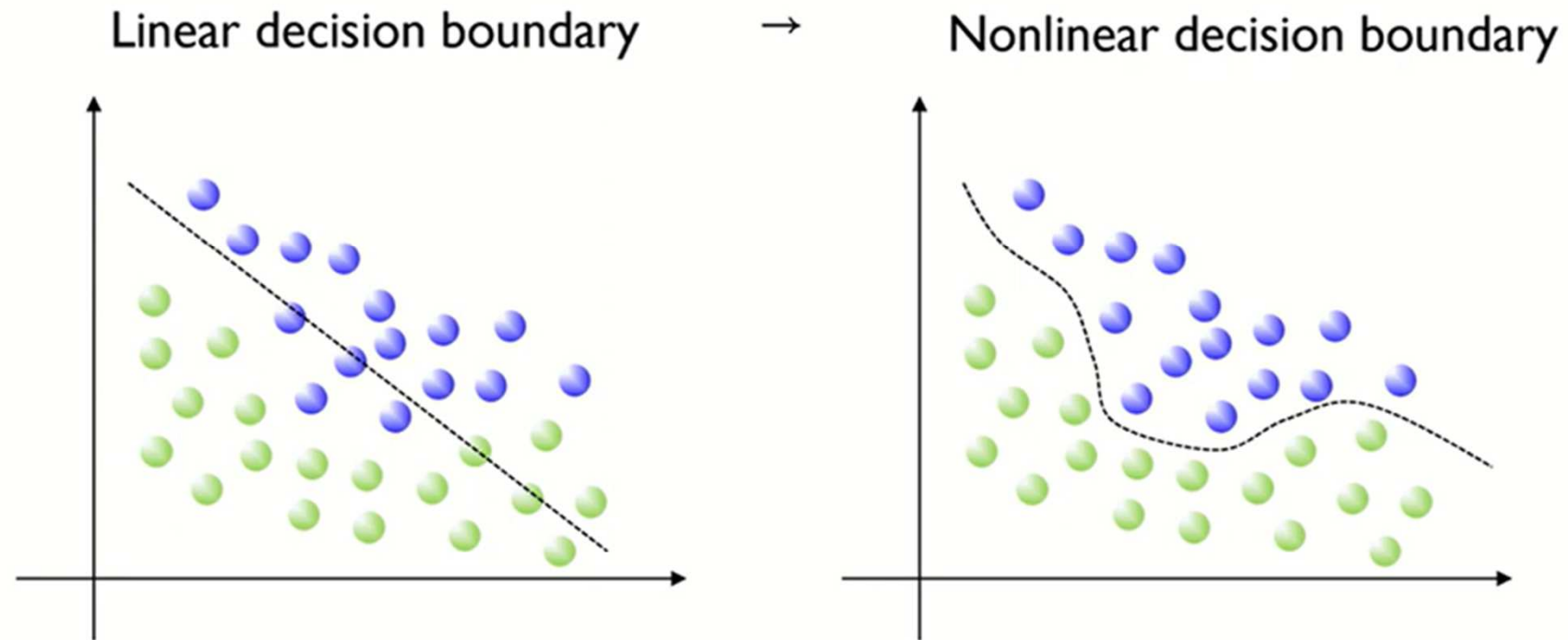
```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from sklearn.svm import SVC
4 from sklearn.datasets import make_blobs
5
6 X, y = make_blobs(n_samples=50, centers=2, random_state=6, cluster_std=1.5)
7 # 라벨을 SVM 수식처럼 -1과 1로 변경
8 y[y == 0] = -1
9
10 # 소프트 마진 (C=0.1) 모델 학습
11 svm_soft = SVC(kernel='linear', C=0.1)
12 svm_soft.fit(X, y)
13
14 # 하드 마진 스타일 (C=1000) 모델 학습
15 svm_hard = SVC(kernel='linear', C=1000.0)
16 svm_hard.fit(X, y)
17
18
19 # 결과 확인: 알파(dual_coef_)의 절대값 출력 (텍스트)
20 def print_alpha_info(model, name): 2개의 사용 위치
21     alphas = np.abs(model.dual_coef_)[0]
22     print(f"[{name}] 결과")
23     print(f"설정된 C 값: {model.C}")
24     print(f"실제 알파(alpha)들의 최대값: {np.max(alphas):.4f}")
25     print(f"C 값에 도달한 서포트 벡터 개수: {np.sum(np.isclose(alphas, model.C))}개")
26     print("-" * 30)
27
28
29 print_alpha_info(svm_soft, name="Soft Margin")
30 print_alpha_info(svm_hard, name="Hard Margin (High C)")
31 fig, axes = plt.subplots(nrows=1, ncols=2, figsize=(14, 6))
32 models = [svm_soft, svm_hard]
33 titles = ["Soft Margin (C=0.1)", "Hard Margin (C=1000.0)"]
34
35 for i, (model, title) in enumerate(zip(models, titles)):
36     ax = axes[i]
37
38     # 데이터 산점도 그리기
39     ax.scatter(X[:, 0], X[:, 1], c=y, cmap='coolwarm', s=50, edgecolors='k')
40
41     # 마진 선을 그리기 위한 가상의 그물망(그리드) 생성
42     xlim, ylim = ax.get_xlim(), ax.get_ylim()
43     xx, yy = np.meshgrid(np.linspace(xlim[0], xlim[1], num=50), np.linspace(ylim[0], ylim[1], num=50))
44     Z = model.decision_function(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)
45
46     # 결정 경계(0) 및 마진 선(-1, 1) 시각화
47     ax.contour(xx, yy, Z, colors='k', levels=[-1, 0, 1], alpha=0.5, linestyles=['--', '-', '--'])
48
49     # 서포트 벡터에 큰 동그라미 그려서 강조
50     ax.scatter(model.support_vectors_[0], model.support_vectors_[1],
51               s=200, linewidth=2, facecolors='none', edgecolors='k')
52
53     ax.set_title(f'Decision Boundary: {title}', size=16)
54
55 plt.tight_layout()
56 plt.show()
```

Soft Margin SVM Classifiers

```
62 def print_equation(model, name): 2개의 사용 위치
63     # 1. w 값 추출 (coef_ 배열에 저장되어 있음)
64     w = model.coef_[0]
65
66     # 2. b 값 추출 (intercept_ 에 저장되어 있음)
67     b = model.intercept_[0]
68
69     print(f"[{name}] 결정 경계 방정식")
70     print(f"w (가중치 벡터): [{w[0]:.4f}, {w[1]:.4f}]")
71     print(f"b (편향/절편): {b:.4f}")
72     # 2차원 데이터이므로  $w_1 \cdot x_1 + w_2 \cdot x_2 + b = 0$  형태가 됩니다.
73     print(f"방정식: {w[0]:.4f} * x1 + {w[1]:.4f} * x2 + {b:.4f} = 0")
74     print("-" * 40)
75
76
77 print_equation(svm_soft, name: "Soft Margin (C=0.1)")
78 print_equation(svm_hard, name: "Hard Margin (C=1000.0)")
```


Kernel Methods for Nonlinear Classification

Nonlinear Decision Boundary



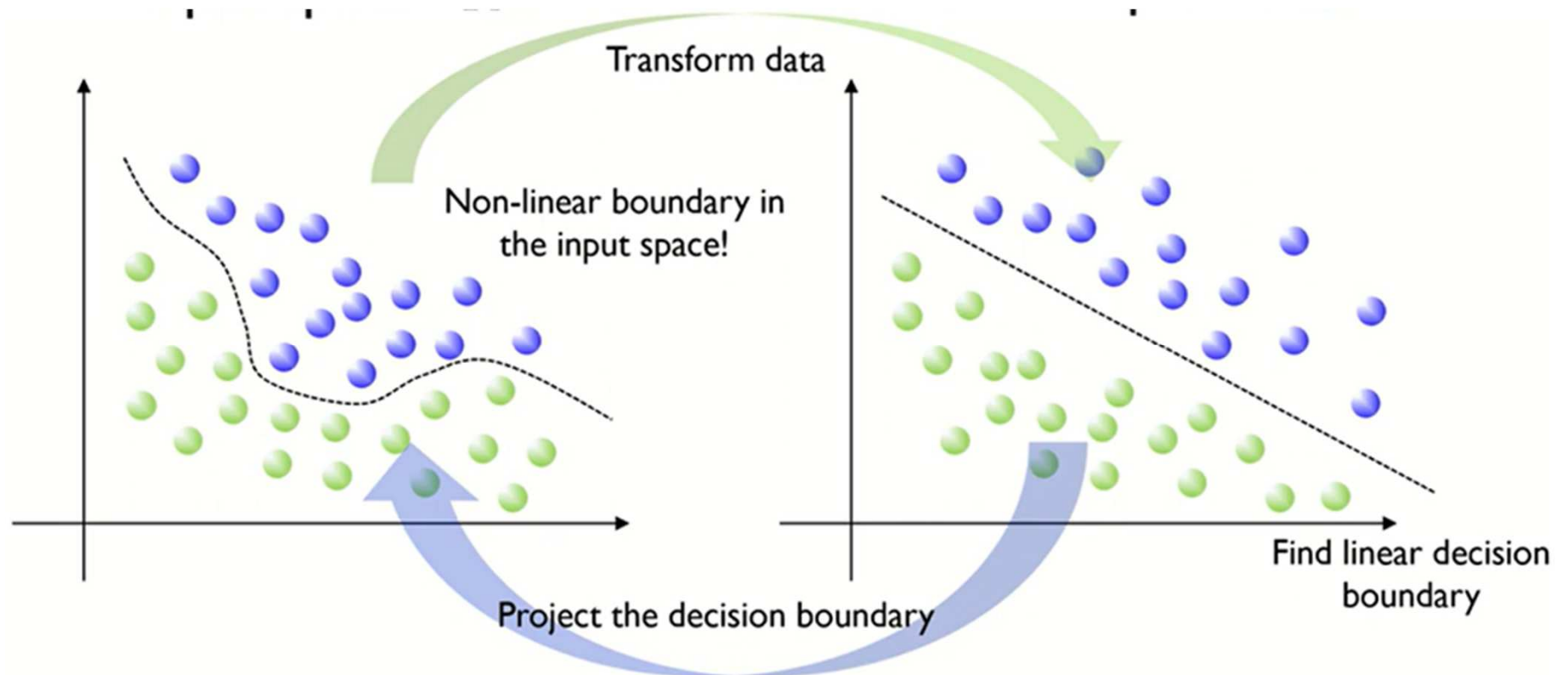
How?

관측치 x 들을 더 높은 차원으로 변환시켜 분류해보자!

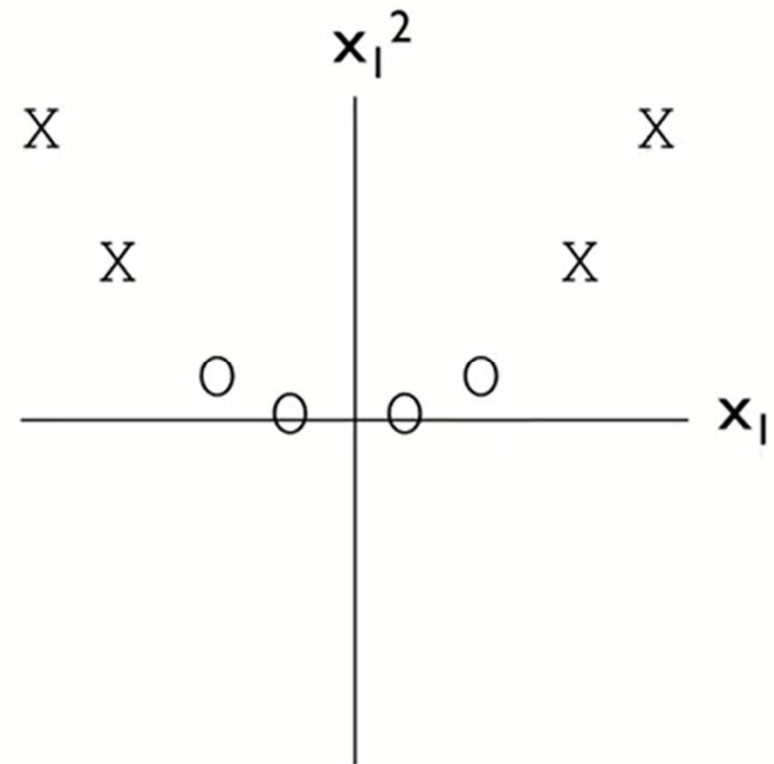
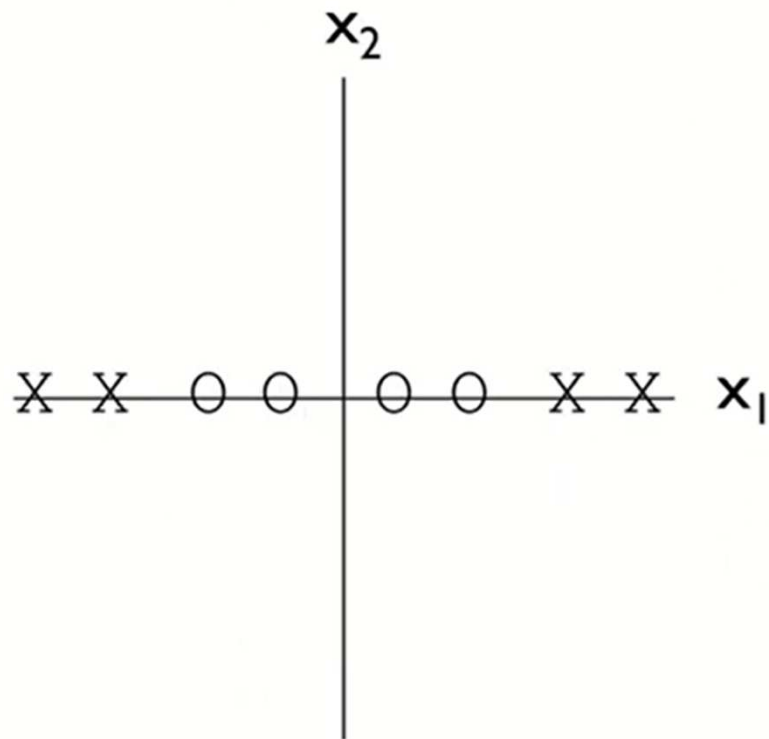
Transforming Data

$$x = (x_1, x_2, \dots, x_n) \rightarrow \phi(x) = z = (z_1, z_2, \dots, z_n)$$

Input Space R^p $p \ll q$ Feature Space R^q



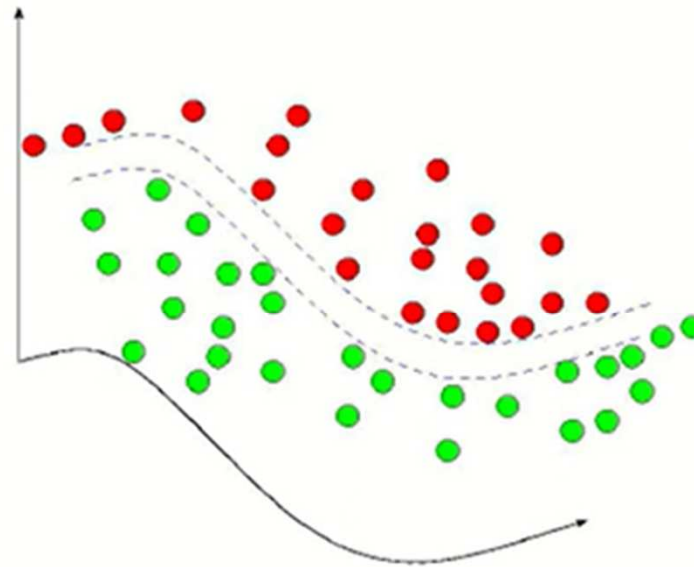
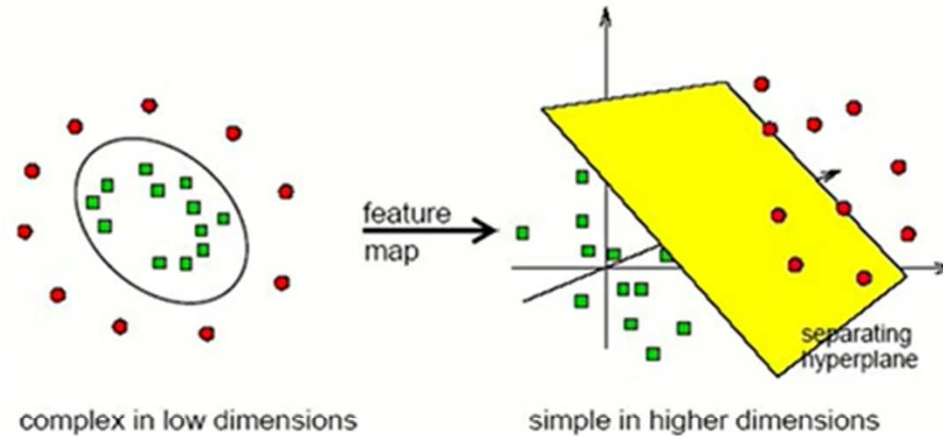
Transforming Data - Example



Transforming Data - Example

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from sklearn.datasets import make_circles
4
5 # 1. 2차원 도넛 모양 데이터 생성
6 X, y = make_circles(n_samples=200, factor=0.3, noise=0.05, random_state=42)
7 x1 = X[:, 0]
8 x2 = X[:, 1]
9
10 # 2. [핵심] 차원 확장 (Kernel Mapping)
11 # 여기서는 중심에서 멀어질수록 값이 커지는  $x_1^2 + x_2^2$  (다항식 커널 형태)를 사용합니다.
12 z = x1**2 + x2**2
13
14 # 3. 2D vs 3D 비교 시각화 세팅
15 fig = plt.figure(figsize=(16, 7))
16
17 # --- [왼쪽 그래프] 원래의 2차원 데이터 ---
18 ax1 = fig.add_subplot(1, 2, 1)
19 ax1.scatter(x1, x2, c=y, cmap='coolwarm', s=50, edgecolors='k')
20 ax1.set_title(label="Original 2D Space\n(Linearly Inseparable)", size=16)
21 ax1.set_xlabel("x1")
22 ax1.set_ylabel("x2")
23
24 # --- [오른쪽 그래프] 확장된 3차원 공간 ---
25 # projection='3d'를 사용하여 3차원 그래프로 설정합니다.
26 ax2 = fig.add_subplot(1, 2, 2, projection='3d')
27 ax2.scatter(x1, x2, z, c=y, cmap='coolwarm', s=50, edgecolors='k')
28
29 # 3차원 공간에서 데이터를 가르는 '평면(Hyperplane)' 그리기
30 # z값이 대략 0.4 정도인 높이에 투명한 평면을 끼워 넣습니다.
31 xx, yy = np.meshgrid(np.linspace(-1.2, 1.2, num=10), np.linspace(-1.2, 1.2, num=10))
32 zz = np.full(xx.shape, fill_value=0.4) # z=0.4 높이의 평면
33 ax2.plot_surface(xx, yy, zz, alpha=0.3, color='gray')
34
35 ax2.set_title(label="Projected 3D Space (z = x1^2 + x2^2)\n(Linearly Separable by a Plane)", size=16)
36 ax2.set_xlabel("x1")
37 ax2.set_ylabel("x2")
38 ax2.set_zlabel("z (New Dimension)")
39
40 # 3D 그래프를 보기 좋은 각도로 돌려줍니다.
41 ax2.view_init(elev=20, azim=45)
42
43 plt.tight_layout()
44 plt.show()
```

Transforming Data - Example



- Changing the representation of the data.
- Use another coordinates system such that the “curve” becomes a “line.”

Mapping Original Space to Kernel Space

$$\phi : x \mapsto z = \phi(x)$$

Example

$$\phi : \underbrace{(x_1, x_2)}_{\substack{\text{2D} \\ \text{(Original Space)}}} \mapsto \underbrace{(x_1, x_2, x_1^2, x_2^2, x_1, x_2)}_{\substack{\text{5D} \\ \text{(Feature Space)}}}$$

- SVM을 original space가 아닌 feature space에서 학습
- Original space에서 nonlinear decision boundary → Feature space에서 linear decision boundary
- 고차원 feature space에서는 관측치 분류가 더 쉬울 수 있음
- 고차원 feature space를 효율적으로 계산할 수 있는 방법이 있음

Kernel Mapping

SVM Lagrangian dual formulation

$$\begin{aligned} & \underset{\alpha}{\text{maximize}} \quad \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j x_i^T x_j \\ & \text{subject to} \quad \sum_{i=1}^n \alpha_i y_i = 0 \\ & \quad \quad \quad 0 \leq \alpha_i \leq C, i = 1, 2, \dots, n \end{aligned}$$

$$\begin{aligned} & \underset{\alpha}{\text{maximize}} \quad \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j \phi(x_i)^T \phi(x_j) \\ & \text{subject to} \quad \sum_{i=1}^n \alpha_i y_i = 0 \\ & \quad \quad \quad 0 \leq \alpha_i \leq C, i = 1, 2, \dots, n \end{aligned}$$

Kernel Mapping

$$\underset{\alpha}{\text{maximize}} \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \alpha_i \alpha_j y_i y_j \phi(x_i)^T \phi(x_j)$$

$$\text{subject to } \sum_{i=1}^n \alpha_i y_i = 0$$

$$0 \leq \alpha_i \leq C, i = 1, 2, \dots, n$$

Inner product of $\phi(x_i)^T \phi(x_j)$
 $\langle \phi(x_i), \phi(x_j) \rangle$

$x_i \rightarrow \phi(x_i)$

$$\underset{\alpha}{\text{maximize}} \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \alpha_i \alpha_j y_i y_j K(x_i, x_j)$$

$$\text{subject to } \sum_{i=1}^n \alpha_i y_i = 0$$

$$0 \leq \alpha_i \leq C, i = 1, 2, \dots, n$$

ϕ 를 이용해서 직접 데이터를 변환할 필요 없이 inner product에 해당하는 $\langle \phi(x_i), \phi(x_j) \rangle$ 만 정의해도 같은 효과를 얻을 수 있음

Kernel Mapping

$$\hat{y}_{new} = \text{sign}(\sum_{i \in SV} \alpha_i^* y_i x_i^T x_{new} + b^*)$$

$x_i \rightarrow \phi(x_i)$

$$\hat{y}_{new} = \text{sign}(\sum_{i \in SV} \alpha_i^* y_i \phi(x_i)^T \phi(x_{new}) + b^*)$$

$$= \text{sign}(\sum_{i \in SV} \alpha_i^* y_i K\langle \phi(x_i), \phi(x_{new}) \rangle + b^*)$$

Kernel Mapping - Example

$$X = (x_1, x_2)$$

$$Y = (y_1, y_2)$$

$$\phi(X) = (x_1^2, x_2^2, \sqrt{2}x_1x_2)$$

$$\phi(Y) = (y_1^2, y_2^2, \sqrt{2}y_1y_2)$$

$$\langle \phi(X), \phi(Y) \rangle = x_1^2y_1^2 + x_2^2y_2^2 + 2x_1x_2y_1y_2$$

Question: Can we compute $x_1^2y_1^2 + x_2^2y_2^2 + 2x_1x_2y_1y_2$ without knowing the explicit functional form of $\phi(X)$ and $\phi(Y)$?

$$\begin{aligned}(X, Y)^2 &= \langle (x_1, x_2), (y_1, y_2) \rangle^2 \\&= \langle x_1y_1 + x_2y_2 \rangle^2 \\&= x_1^2y_1^2 + x_2^2y_2^2 + 2x_1x_2y_1y_2 \\&= \langle \phi(X), \phi(Y) \rangle\end{aligned}$$

→ This can be obtained without knowing the explicit functional form of $\phi(X)$ and $\phi(Y)$

without knowing the explicit = implicit

$$(X, Y)^2 = \langle (x_1, x_2), (y_1, y_2) \rangle^2 = K(X, Y) \text{ (Kernel function)}$$

Kernel Functions

- Linear kernel

$$K\langle x_1, x_2 \rangle = \langle x_1, x_2 \rangle$$

- Polynomial kernel

$$K\langle x_1, x_2 \rangle = (a\langle x_1, x_2 \rangle + b)^d$$

- Sigmoid kernel (Hyperbolic tangent kernel)

$$K\langle x_1, x_2 \rangle = \tanh(a\langle x_1, x_2 \rangle + b)$$

- Gaussian kernel (Radial basis function (RBF) kernel)

$$K\langle x_1, x_2 \rangle = \exp\left(\frac{-\|x_1 - x_2\|_2^2}{2\sigma^2}\right)$$

Choosing Kernel Functions

- SVM 사용시 kernel을 결정하는 것은 어려운 문제 → 딱히 기준이 없음
- 사용하는 kernel에 따라 feature space의 특징이 달라지기 때문에 데이터의 특성에 맞는 kernel을 결정하는 것은 중요함
- 일반적으로는 RBF kernel, sigmoid kernel, low degree polynomial kernel 또는 등이 주로 사용됨

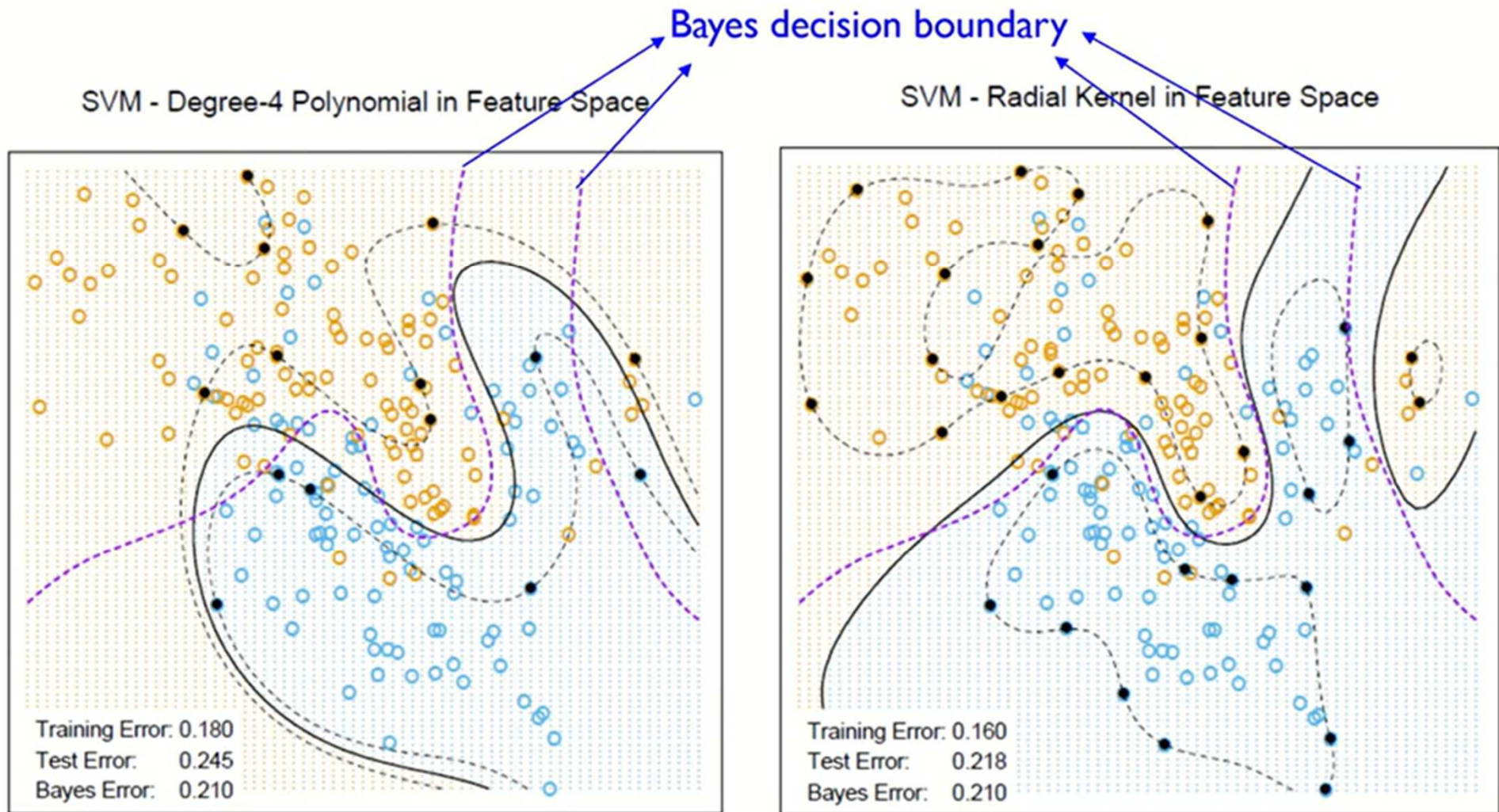
Choosing Kernel Functions

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from sklearn.svm import SVC
4 from sklearn.datasets import make_circles
5 from sklearn.model_selection import train_test_split
6 from sklearn.metrics import accuracy_score
7
8 X, y = make_circles(n_samples=300, factor=0.3, noise=0.15, random_state=42)
9
10 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
11
12 #3가지 모델 정의
13 models = {
14     "1. Linear Kernel ": SVC(kernel='linear', C=1.0),
15     "2. RBF Kernel ": SVC(kernel='rbf', C=1.0, gamma=0.5),
16     "3. RBF Kernel ": SVC(kernel='rbf', C=1.0, gamma=50.0)
17 }
18 ESTIMATED_BAYES_ERROR = 0.05
19 print("=" * 50)
20 print(f" [이론적 한계] Bayes Error (Irreducible Error): 약 {ESTIMATED_BAYES_ERROR * 100:.1f}%\n"
21       f"      (데이터 생성 시 부여된 15%의 노이즈로 인해 발생하는 본질적 결함)")
22 print("=" * 50)
23 # 모델 학습 및 Error 계산
24 fig, axes = plt.subplots(nrows=1, ncols=3, figsize=(18, 5))
25 for i, (title, model) in enumerate(models.items()):
26     # 학습
27     model.fit(X_train, y_train)
28     # 예측
29     y_train_pred = model.predict(X_train)
30     y_test_pred = model.predict(X_test)
31     # Error Rate 계산 (1 - 정확도)
32     train_error = 1.0 - accuracy_score(y_train, y_train_pred)
33     test_error = 1.0 - accuracy_score(y_test, y_test_pred)
34     # 결과 출력
35     print(f"[{title}]")
36     print(f" - Train Error: {train_error * 100:.1f}%")
37     print(f" - Test Error : {test_error * 100:.1f}%")
38     print("-" * 50)
39     # === 시각화 (결정 경계 및 Test 데이터 산점도) ===
40     ax = axes[i]
41     # 훈련 데이터는 연하게, 테스트 데이터는 진하게 표시
42     ax.scatter(X_train[:, 0], X_train[:, 1], c=y_train, cmap='coolwarm', s=20, alpha=0.3)
43     ax.scatter(X_test[:, 0], X_test[:, 1], c=y_test, cmap='coolwarm', s=60, edgecolors='k', marker='^')
44
45     xlim, ylim = ax.get_xlim(), ax.get_ylim()
46     xx, yy = np.meshgrid(np.linspace(xlim[0], xlim[1], num=100), np.linspace(ylim[0], ylim[1], num=100))
47     Z = model.decision_function(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)
48
49     ax.contour(xx, yy, Z, colors='k', levels=[0], alpha=0.8, linestyle=['-'])
50     ax.set_title(f"[{title}]\nTest Err: {test_error * 100:.1f}%", size=13)
51
52 plt.tight_layout()
53 plt.show()
```


Choosing Kernel Functions

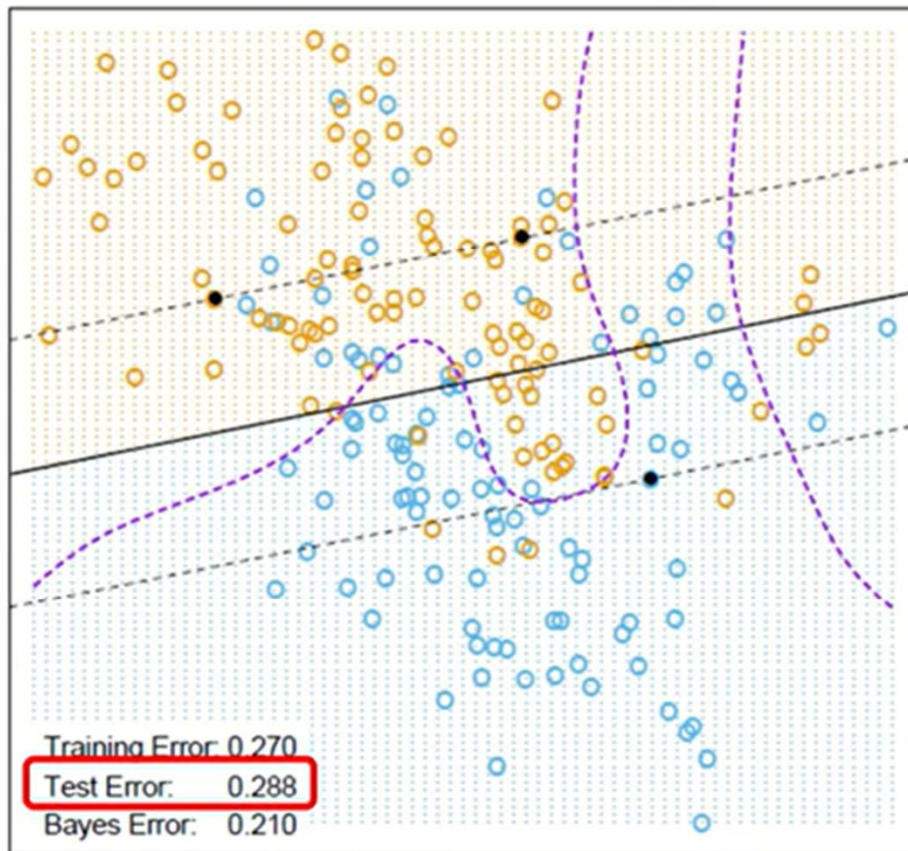
```
55 # 딕셔너리의 모델(Value)들만 싹 모아서 리스트로 바꾼 뒤, 첫 번째[0] 모델을 가져옵니다.
56 linear_model = list(models.values())[0]
57 |
58 print("\n[수학적 증명: 선형 SVM의 항복 선언]")
59 print("선형 SVM의 가중치(w):", linear_model.coef_)
60 print("선형 SVM의 편향(b):", linear_model.intercept_)
```

Nonlinear(Kernel) SVM Classifiers



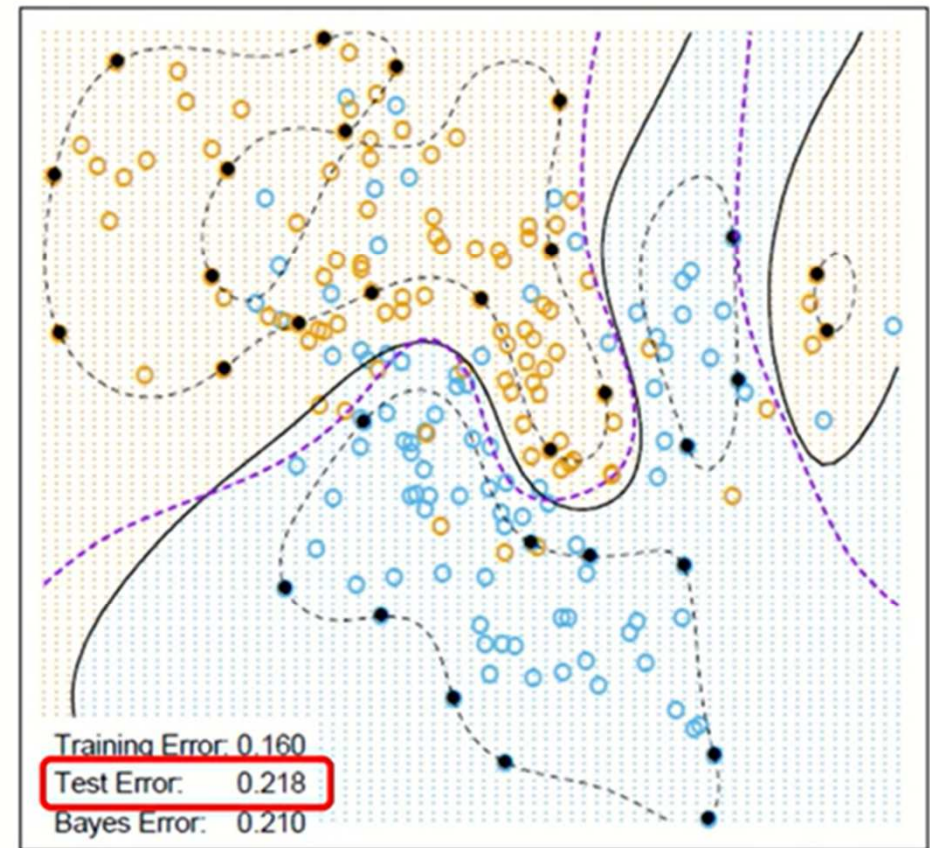
Linear vs Nonlinear SVM Classifiers

Soft Margin **Linear** SVM Classifiers



$C = 10000$

Soft Margin **Nonlinear** SVM Classifiers



SVM - Radial Kernel in Feature Space